

Inigo Quilez :: articles :: how to render in VR - 2016



Intro

Over the years I've written a few demos and engines that work in VR (mostly in the mid 2000s and again now in the mid 2010s). Since it's a hot topic again, I have had to explain to different people in different emails to different degrees of detail how I design my VR demos/engine. So, I thought it might be a good idea to stop doing so and instead drop here some basic advices and psuedocode that makes the points I usually try to get across. Now, this is how I do VR, not necesarily how my employer or anybody else than me recommends it to be done. Also, I won't be discusing advanced topics such as multi GPU and multi context (obsolete since the death of SGI), late latching or foveated rendering.

Now, lets start!

Basics

Before I describe my engine design, some basic things you should take into account that you might miss if you come from a traditional engine design:

- **Camera != Eye.** Usually shaders do computations based on eye/camera position (such as speculars, fresnel, LOD, etc). In VR, camera and eye are two different concepts, and you should not fuse them, but expose them as two separate pieces of data in your uniforms. The eye position is the apex of your projection frustum, and that's what you use for doing speculars, fresnets, etc. The camera is where your viewer's head is in space, and it's what you use to do Level of Detail (LOD), geographical computations, viewer based fades, etc.
- **Do not mess with scales.** Do not modify the scale of the world, do not amplify the camera movements. If you are not having good VR it means (a) you are not modeling your world after real units (b) you got

the VR math wrong. If you start hacking scenes or head scales, you are going to be in an ugly mess very quickly. For the same reason ad-hock shading models have been replaced by Physically Based Shading and Materials, you want your VR to be Physically Based, not random-scale-factors-based.

A good indication that your VR is wrong is when you put a half a meter cube spinning at one meter in front of you and people cannot tell you, unambiguously, how big and how far the cube is. If they cannot put their hands exactly where it is you are most probably doing VR wrong (or they are stereo blind, which happens every now and then)

- **VR != Stereo.** Don't assume you have only two render passes which have overlapping frustums. In the past VR was done with multiple projections walls, and perhaps one day in the future headsets will have peripheral displays (perhaps at lower resolution than the central ones, perhaps mono rendered instead of stereo - and this is a pure uninformed fantasy of mine, but it costs little to plan for this).
- **Context.** Soon enough you'll see that in VR you have variables that are constant across the demo/game, across the frame, across the display, and across the eye-render. Decide which uniforms belong to each refresh-group and create uniform blocks for each. I like not putting them all in one single block for clarity, especially when writing shaders, where you need to decide from which context you are going to pull a transformation matrix for example.

Design

Ok, now we are ready to have a look to how I have been organizing my VR engines and demos so far:

```

ComputeFrame( k )
{
    DoProcessFrame( k );           // Compute animation, sound, LOD comp

    DoRenderFrame( k );           // Render reflection maps, clouds lay

    for( int j=0; j < numDisplays; j++ ) // numDisplays = 5 for a CAVE, 1 for
    {
        DoProcessEye( k, j );      // Do frustum culling, compile draw c

        DoRenderEyeCommon( k, j ); // Render eye-shared shadow maps, and

```

```

for( int i=0; i < 2; i++ )           // stereo
{
    DoRenderEye( k, j, i );           // Render regular geometry and postp
}
}

```

This is the basic structure. The three contexts (frame, display and eye) are represented by **k**, **j** and **i**. The tasks that happen in each one of them is not totally rigid, and it depends on the specifics of the target configuration. For example, in VR based on Head Mounted Devices, LOD or Frustum culling can be done at frame granularity, while in a CAVE or systems with multiple displays (disjoining frustums) it might be worth doing it at a per display level. Also, depending on your engine, you might decide to do your culling in the GPU rather than the CPU. But overall, the structure of the pseudocode above is pretty useful.

- At **frame** level (**k**), you do your constant computations. That includes posing the worlds animated characters, but also some rendering related tasks such as computing global shadow maps, reflection maps, sky animated textures and other things you might need later that are view orientation independent. You might want to first collect all of the frustums that will eventually be used for rendering, make a union operation, and cull objects in the scene to that volume before doing these rendering. That list of objects can be passed down the pipeline too. In the case of domestic VR based on a modern Head Mounted Device, this global frustum is simply the union of your left and right eye frustums. Most APIs will give you these in the form of 4 clipping planes per eye. In more advanced VR setups the frustums can be as many as 12 and it might not be useful to do any culling.

Things you want to put in your "Frame (**k**)" uniform/constant buffer for shaders to use later down the pipe include the camera position (remember, not the same as the eye position), the time, the delta time, etc.

- At **display** level (**j**), you do your usual frustum culling. This is the context right before the stereo rendering, so both eye's frustum are going to overlap a lot, which is great. You might want to collect the objects that pass the test and compile here your indirect render calls or command buffers, so you don't have to submit render calls twice or use fancy extensions.

You can also do rendering work than can be shared across left and right eye but not across displays. For example, the local shadow maps. For the main render, you can also render all the distant objects into a texture that you can composite later in the stereo renders. Objects that are so far that have sub-pixel parallax are good candidates, and can help you save lots of rendering time.

Things you can put in your "Display (j)" shader uniform block are the display projection matrix and position, the display resolution (do not put it in the Frame context just in case you have displays with varying resolutions - remember peripheral displays), etc.

- At **eye** level (i) you do the rendering of the objects close to the viewer that need parallax. This is what resembles your old-school rendering code the most, so go wild with your fancy demo.

This time in the "Eye (i)" shader constant block you can put the actual eye camera matrix and projection so you can do correct lighting and shading.

You might be thinking that rendering objects at the Display (j) level and Eye (i) level might create one extra doubling of shader permutations since they might have to pull data from different uniform blocks (one to get the camera position for the display - somewhere near the center of both eye - and the other from the actual eye) to do proper lighting. You can solve this maybe with an extra uniform block that aliases one or the other somehow.

Conclusions

I cannot go in much more detail in this article, but if you want to see lower level implementation details to make this work fast, check [this article on fast stereo rendering](#). Otherwise, hopefully you still got a big picture idea of the changes you might need to apply to your old mono-single-display-non-vr-boring-2d engine :)

[inigo quilez](#) - learning computer graphics since 1994